학습 교안

경사하강법
기본 수식 유도하기

오차 · MSE · 기울기 · 편미분과 연쇄법칙 · 학습률 · w와 b 업데이트

항목	내용
원본 영상	테디노트 TeddyNote, 「경사하강법 (Gradient Descent)의 기본 수식에 대해 알려드립니다」 (17분 42초, 2020.05.13)
영상 주소	https://www.youtube.com/watch?v=GEdLNvPIbiM
대상	머신러닝을 처음 배우며 '경사하강법이 수식으로 어떻게 나오는지' 알고 싶은 학생
선수 지식	Σ · 평균 · 1차 함수 · 미분 · 편미분 · 연쇄법칙 (「ML_기초수학_학습교안」 1~9장)
학습 시간	영상 시청 18분 + 교안 정리 및 실습 약 2시간
준비물	Python 3, numpy, sympy, matplotlib (실습 코드 폴더의 README 참고)

학생 배포용 · 영상 내용을 바탕으로 정리하고 실습 코드를 덧붙인 교안입니다.

목차

0장 들어가며 . . . 3

1장 오차와 손실 함수 . . . 4

2장 경사하강법의 아이디어 . . . 7

3장 손실 함수를 w, b 의 식으로 . . . 9

4장 w 에 대한 편미분 . . . 10

5장 학습률과 업데이트 . . . 12

6장 b 에 대한 편미분 . . . 13

7장 최종 정리와 파이썬 구현 . . . 14

8장 학습률 실험과 최소제곱법 (보충) . . . 16

부록 A 확인 문제 정답 및 해설 . . . 18

부록 B 공식 한눈에 보기 . . . 19

CHAPTER 0

들어가며

머신러닝 모델은 처음에는 엉터리 답을 내놓습니다. 그런데 데이터를 보며 조금씩 스스로를 고쳐서 점점 정답에 가까워집니다. 이 "조금씩 고치는 방법"의 가장 기본이 바로 **경사하강법(Gradient Descent)**입니다.

영상은 화이트보드에 직접 손으로 수식을 써 가며 **오차 → 손실 함수(MSE) → w, b 에 대한 편미분 → 학습률 → 업데이트 식**까지 한 번에 유도합니다. 이 교안은 그 흐름을 그대로 따라가며 판서 내용을 깔끔한 수식으로 다시 쓰고, 영상에서 "다음 시간에"로 미룬 파이썬 구현과 학습률 실험을 7·8장에 덧붙였습니다.

교안 구성

장	주제	영상 구간	한 줄 요약
1	오차와 손실 함수	00:38~03:57	오차 $\hat{y} - y$, 제곱해서 평균 낸 MSE
2	경사하강법의 아이디어	03:57~05:13	그릇 바닥(기울기 0)을 향해 걸어 내려가기
3	손실 함수를 w, b 의 식으로	05:13~07:20	$L(w, b) = (1/N)\Sigma(wx + b - y)^2$
4	w 에 대한 편미분	07:20~09:30	연쇄법칙: 겉 미분 $2(\cdot)$ × 속 미분 x
5	학습률과 업데이트	09:30~12:50	LR 을 곱해서 조금씩 빼 준다
6	b 에 대한 편미분	12:50~15:10	속 미분이 1 이라 더 간단
7	최종 정리와 파이썬 구현	15:10~끝	업데이트 식 두 줄 → 코드 두 줄
8	학습률 실험과 최소제곱법 (보충)	-	학습률이 크면 발산, 정답은 OLS 와 일치

공부하는 방법

1) 각 장의 **영상 구간**을 보고, 2) 교안의 수식을 **직접 손으로 따라 써 보세요**. 이 영상은 유도 과정 자체가 핵심이라 눈으로만 보면 금방 놓칩니다. 3) 실습 코드로 수식이 맞는지 숫자로 확인하고, 4) 확인 문제를 풀고 부록 A 와 맞춰 봅니다.

핵심 포인트

- 편미분과 연쇄법칙이 낯설다면 「ML_기초수학_학습교안」 8·9장을 먼저 복습하세요.
- 모든 실습 결과는 실제로 실행한 값입니다. 데이터는 $y = 3x + 2$ 에 잡음을 섞어 만든 30개입니다.

CHAPTER 1

오차와 손실 함수

영상 구간 **00:38~03:57** 오차의 정의, 부호 상쇄 문제, MSE · MAE, 손실 함수 L

학습 목표

- 예측값 ŷ 와 실제값 y 로 오차를 정의할 수 있다.
- 오차를 그냥 더하면 안 되는 이유를 설명할 수 있다.
- SSE, MSE, MAE 를 구분하고 손실 함수의 의미를 이해한다.

1.1 오차 = 예측값 - 실제값

데이터 점(실제값 y)들 사이로 직선 ŷ = ax + b 를 하나 그었다고 합시다. 직선 위의 값이 **예측값** ŷ이고, 점과 직선 사이의 세로 거리가 **오차**입니다. 영상에서는 오차를 다음과 같이 정의합니다.

$$\text{error} = \hat{y} - y$$

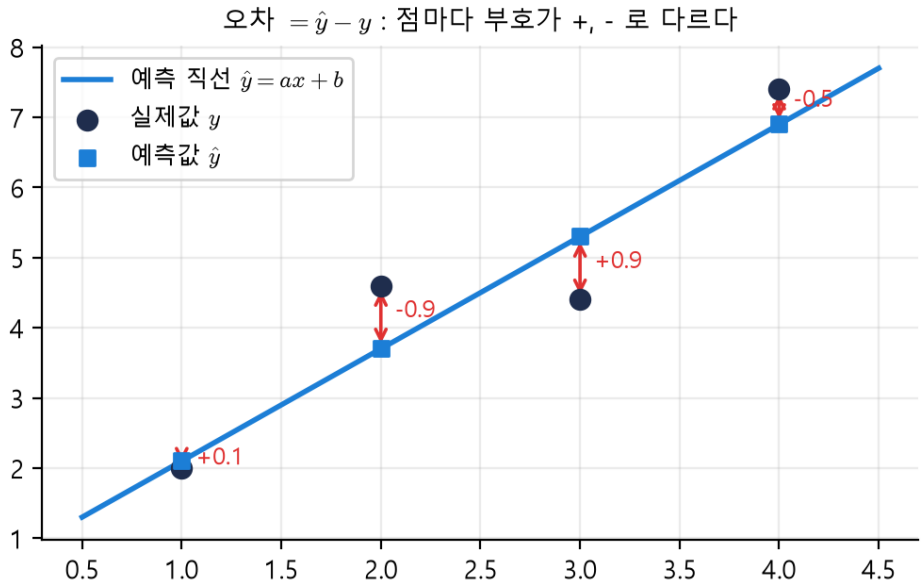


그림 1-1. 점마다 오차가 있고, 직선보다 위에 있는 점과 아래에 있는 점의 오차 부호가 다르다

1.2 그냥 더하면 안 되는 이유

전체 오차를 알고 싶어 Σ(ŷ - y) 를 구하면, +1 과 -1 같은 값이 서로 **상쇄**되어 모델이 엉망이어도 0 이 나올 수 있습니다. 그래서 부호를 없애기 위해 **제곱**하거나 **절댓값**을 씌웁니다.

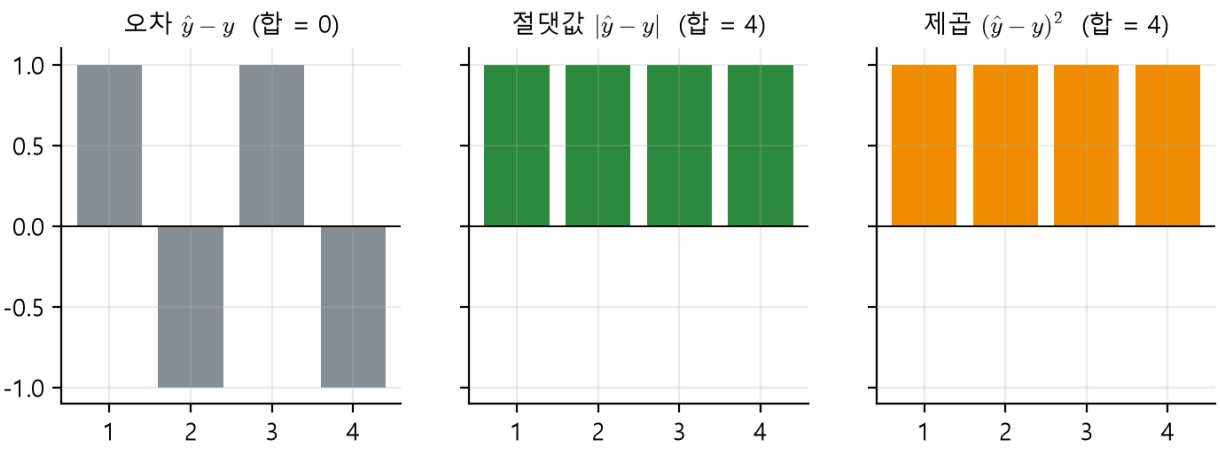


그림 1-2. 오차 [+1, -1, +1, -1] 은 더하면 0 이지만, 절댓값이나 제곱은 0 이 되지 않는다

이름	식	설명
SSE (제곱 오차의 합)	$\sum(\hat{y}_i - y_i)^2$	데이터가 많을수록 값이 한없이 커진다
MSE (평균 제곱 오차)	$\frac{1}{N}\sum(\hat{y}_i - y_i)^2$	SSE 를 개수 N 으로 나눈 평균. **이 교안에서 사용**
MAE (평균 절대 오차)	$\frac{1}{N}\sum\|\hat{y}_i - y_i\|$	절댓값의 평균. 큰 오차에 덜 민감

영상에서는 **미분하기 편하도록 MSE** 를 사용합니다. (절댓값 |u| 는 u = 0 에서 뾰족하게 꺾여 미분이 정의되지 않지만, 제곱 u² 는 어디서나 매끄럽게 미분됩니다.)

1.3 손실 함수 (Loss Function)

MSE 처럼 "모델이 얼마나 틀렸는지"를 하나의 숫자로 알려 주는 함수를 **손실 함수(loss function)** 또는 비용 함수(cost function)라고 하며, 보통 **L** 로 씁니다. **학습의 목표는 L 을 가장 작게 만드는 것**입니다.

$$L = \text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

주의

- 책이나 강의에 따라 오차를 y - ŷ 로 쓰기도 합니다. 제곱하면 (ŷ - y)² = (y - ŷ)² 이므로 MSE 는 똑같고, 미분할 때 부호만 바뀌어 **최종 업데이트 결과는 같습니다**. 한 가지로 정해서 끝까지 일관되게 쓰는 것이 중요합니다.

실습 코드 0 - 준비 (이후 모든 실습에서 사용할 데이터)

```
import numpy as np
import sympy as sp
np.set_printoptions(precision=4, suppress=True)

# 실습 데이터: 정답 규칙 y = 3x + 2 에 약간의 잡음을 섞은 30개
rng = np.random.default_rng(42)
X = rng.uniform(0, 5, 30)
Y = 3 * X + 2 + rng.normal(0, 0.5, 30)
print("X 앞 5개:", X[:5])
print("Y 앞 5개:", Y[:5])
```

실행 결과

```
X 앞 5개: [3.8698 2.1944 4.293  3.4868 0.4709]
Y 앞 5개: [14.6802  8.38   14.6228 12.0536  3.7206]
```

실습 코드 1-1 - 오차의 합 vs SSE · MSE · MAE

```
y     = np.array([3.0, 5.0, 7.0, 9.0])     # 실제값
y_hat = np.array([4.0, 4.0, 8.0, 8.0])     # 예측값

err = y_hat - y                            # 영상의 오차 정의: y hat - y
print("오차 y_hat - y :", err)
print("그냥 더하면    :", err.sum(), "  <- +, - 가 상쇄되어 0!")
print("SSE (제곱 합)  :", np.sum(err ** 2))
print("MSE (제곱 평균):", np.mean(err ** 2))
print("MAE (절댓값 평균):", np.mean(np.abs(err)))
```

실행 결과

```
오차 y_hat - y : [ 1. -1.  1. -1.]
그냥 더하면    : 0.0   <- +, - 가 상쇄되어 0!
SSE (제곱 합)  : 4.0
MSE (제곱 평균): 1.0
MAE (절댓값 평균): 1.0
```

핵심 포인트

- 오차 = ŷ - y. 그냥 더하면 상쇄되므로 제곱(또는 절댓값).
- MSE = 제곱 오차의 평균 = 손실 함수 L. 학습 = L 줄이기.

확인 문제

Q1. y = [2, 4, 6], ŷ = [3, 3, 7] 일 때 오차(ŷ - y), 오차의 합, SSE, MSE, MAE 를 각각 구하시오.
Q2. SSE 대신 MSE(평균)를 쓰면 좋은 점은 무엇인가?

CHAPTER 2

경사하강법의 아이디어

영상 구간 **03:57~05:13** MSE 그래프는 둥근 모양, 기울기가 0 인 곳을 찾아 이동

학습 목표

- 손실 함수 그래프에서 최솟값의 위치와 기울기의 관계를 설명할 수 있다.
- 기울기의 부호로 이동 방향을 정할 수 있다.

가중치 w 를 바꿔 가며 MSE 를 그려 보면 아래로 볼록한 **그릇 모양**이 나옵니다. 오차가 가장 작은 곳은 그릇의 **가장 낮은 점**이고, 그곳에서 접선의 **기울기는 0** 입니다.

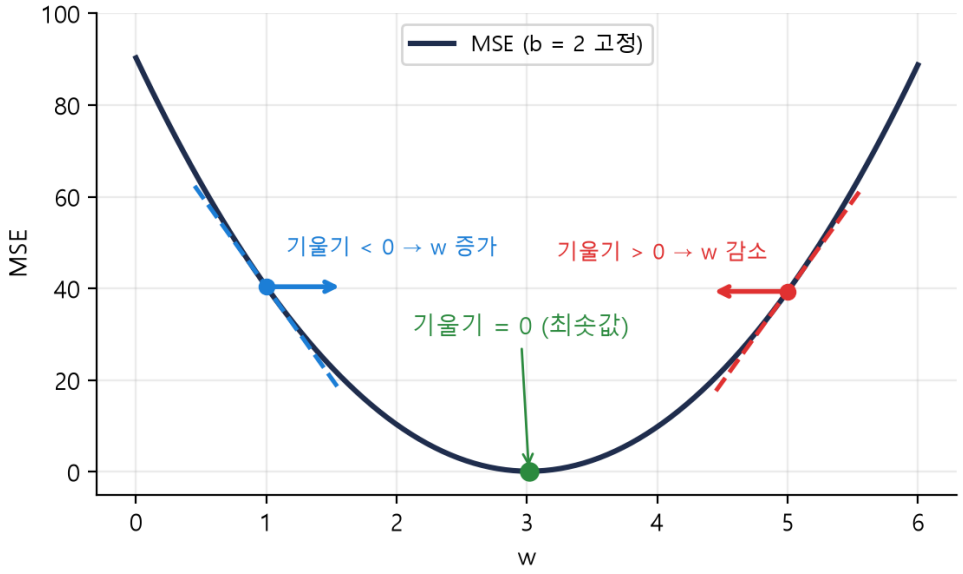


그림 2-1. 실습 데이터의 MSE 곡선 (b = 2 고정). 기울기의 부호가 가야 할 방향을 알려 준다

경사하강법은 눈을 가린 채 산을 내려가는 것과 같습니다. 발밑의 경사만 느끼면서 **내리막 쪽으로 한 걸음씩** 옮기는 것이죠.

단계	하는 일
① 시작	w 를 아무 값(랜덤)으로 정한다
② 기울기 계산	현재 위치에서 손실 함수의 기울기(미분값)를 구한다
③ 이동	기울기의 **반대 방향**으로 조금 이동한다
④ 반복	기울기가 0 에 가까워질 때까지(더 내려갈 곳이 없을 때까지) ②~③ 을 반복한다

현재 기울기	의미	w 를 어떻게?
음수 (-)	오른쪽으로 갈수록 손실이 줄어든다 (최솟값이 오른쪽)	w 를 **키운다**
양수 (+)	왼쪽으로 갈수록 손실이 줄어든다 (최솟값이 왼쪽)	w 를 **줄인다**

현재 기울기	의미	w 를 어떻게?
0	바닥에 도착	멈춘다

두 경우 모두 **"w 에서 기울기를 빼면"** 올바른 방향으로 움직입니다. 기울기가 음수면 빼는 것이 곧 더하는 것이기 때문입니다. 그래서 업데이트 식이 w ← w - (기울기) 모양이 됩니다.

실습 코드 2-1 - w 를 바꿔 가며 손실 재 보기

```
def mse(w, b):
    Y_hat = w * X + b
    return np.mean((Y_hat - Y) ** 2)

# b = 2 로 고정하고 w 만 바꿔 가며 손실을 재 보기
for w in [0, 1, 2, 2.5, 3, 3.5, 4, 5]:
    print(f"w = {w:3.1f}  ->  MSE = {mse(w, 2):8.4f}")
```

실행 결과

```
w = 0.0  ->  MSE =  90.3137
w = 1.0  ->  MSE =  40.3986
w = 2.0  ->  MSE =  10.3473
w = 2.5  ->  MSE =   2.7705
w = 3.0  ->  MSE =   0.1597
w = 3.5  ->  MSE =   2.5149
w = 4.0  ->  MSE =   9.8359
w = 5.0  ->  MSE =  39.3759
```

w = 3 근처에서 MSE 가 가장 작고, 양쪽으로 멀어질수록 커집니다. 정답 규칙(y = 3x + 2)과 일치하죠. 하지만 실제 모델에는 파라미터가 수백만 개라 이렇게 다 넣어 보는 방법은 쓸 수 없습니다. 그래서 기울기를 이용합니다.

핵심 포인트

- 손실 곡선의 바닥 = 기울기 0 = 가장 좋은 w.
- w ← w - 기울기 : 기울기 부호와 상관없이 항상 내리막으로 간다.

확인 문제

Q1. 현재 w 에서 손실 함수의 기울기가 +12 이다. w 를 키워야 하는가, 줄여야 하는가?
Q2. 그릇 모양 손실 함수에서 기울기가 -5 인 점과 -0.5 인 점 중 바닥에서 더 멀리 있는 점은? 이유는?

CHAPTER 3

손실 함수를 w, b 의 식으로

영상 구간 **05:13~07:20** a → w(weight), ŷ = wx + b 를 MSE 에 대입

학습 목표

- ŷ = wx + b 를 MSE 에 대입해 L(w, b) 를 쓸 수 있다.
- 학습할 때 '변수'와 '상수'가 무엇인지 구분할 수 있다.

기울기를 0 으로 만들려면 미분을 해야 하고, 이제부터 **편미분**과 **연쇄법칙(chain rule)**이 필요합니다. 먼저 식을 정리합니다. 머신러닝에서는 기울기 a 를 **가중치(weight) w**, 절편 b 를 **편향(bias) b** 로 씁니다.

$$\hat{y} = ax + b \quad \longrightarrow \quad \hat{y} = wx + b$$

이 ŷ 를 MSE 식에 그대로 넣으면(대입), 손실 함수가 w 와 b 로 표현됩니다.

$$L = \frac{1}{N}\sum(\hat{y} - y)^2 \quad \Longrightarrow \quad L(w, b) = \frac{1}{N}\sum_{i=1}^{N}(wx_i + b - y_i)^2$$

관점 바꾸기: 무엇이 변수인가?

평소에 y = wx + b 를 보면 x 가 변수였습니다. 하지만 **학습할 때는 반대**입니다.

- x, y : 이미 주어진 **데이터** → 바뀌지 않으므로 **상수**
- w, b : 우리가 찾아야 하는 값 → **변수**

그래서 L 을 w 로 미분하고, b 로 미분합니다. 변수가 2 개이므로 **편미분**입니다.

핵심 포인트

- L(w, b) = (1/N)Σ(wx + b - y)²
- 학습에서 변수는 w, b. 데이터 x, y 는 상수 취급.

확인 문제

Q1. L(w, b) = (1/N)Σ(wx + b - y)² 에서 w 로 편미분할 때 상수로 취급되는 것을 모두 쓰시오.

Q2. 데이터 (x, y) = (2, 7) 하나, w = 3, b = 0 일 때 ŷ 와 제곱 오차 (ŷ - y)² 를 구하시오.

CHAPTER 4

w 에 대한 편미분

영상 구간 **07:20~09:30** ∂/∂w, 바깥 껍데기부터 미분 → 안쪽 미분 → w 의 그래디언트

학습 목표
- 연쇄법칙으로 ∂L/∂w 를 유도할 수 있다.
- 유도한 식이 맞는지 코드로 확인할 수 있다.

영상의 설명대로 **바깥 껍데기(제곱)부터** 미분하고, 그다음 **안쪽**을 미분해서 곱합니다. 안쪽을 u = wx + b - y 로 놓으면 L 은 u^2 의 평균입니다.

단계	계산	설명
① 상수 1/N 은 밖으로	$\frac{\partial L}{\partial w} = \frac{1}{N}\sum \frac{\partial}{\partial w}(wx+b-y)^2$	Σ 와 1/N 은 미분에 영향 없음
② 겉 미분	$(u^2)' = 2u = 2(wx+b-y)$	지수 2 가 앞으로 내려옴
③ 속 미분	$\frac{\partial}{\partial w}(wx+b-y) = x$	w → 1, b·y 는 상수 → 0
④ 곱하기	$\frac{\partial L}{\partial w} = \frac{1}{N}\sum 2(wx+b-y)\cdot x$	겉 미분 × 속 미분

마지막으로 상수 2 를 Σ 밖으로 꺼내고, wx + b 를 다시 ŷ 로 바꿔 쓰면 깔끔해집니다.

$$\frac{\partial L}{\partial w} = \frac{2}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)\,x_i = 2\cdot \mathrm{MEAN}((\hat{y}-y)\cdot x)$$

이 값이 w 의 **그래디언트(gradient)**, 즉 "w 를 얼마나 바꿔야 하는지"를 알려 주는 기울기입니다. 의미를 풀어 보면 **(오차 × 입력값)의 평균**입니다. 오차가 크고 입력 x 가 큰 데이터일수록 w 를 크게 고치라는 뜻이죠.

실습 코드 4-1 - sympy 로 데이터 1개의 편미분 확인

```
w, b, x, y = sp.symbols('w b x y')
loss = (w*x + b - y)**2                    # 데이터 1개의 제곱 오차

print("∂/∂w :", sp.factor(sp.diff(loss, w)))
print("∂/∂b :", sp.factor(sp.diff(loss, b)))
```

실행 결과

```
∂/∂w : 2*x*(b + w*x - y)
∂/∂b : 2*(b + w*x - y)
```

실습 코드 4-2 - 공식으로 구한 기울기 vs 숫자로 잰 기울기

```
def grad(w, b):
    Y_hat = w * X + b
    dw = 2 * np.mean((Y_hat - Y) * X)      # ∂MSE/∂w
    db = 2 * np.mean(Y_hat - Y)            # ∂MSE/∂b
    return dw, db

w, b, h = 1.0, 0.0, 1e-6
num_dw = (mse(w + h, b) - mse(w - h, b)) / (2 * h)    # 숫자로 잰 기울기
num_db = (mse(w, b + h) - mse(w, b - h)) / (2 * h)
print("공식 dw, db :", np.round(grad(w, b), 4))
print("수치 dw, db :", np.round([num_dw, num_db], 4))
```

실행 결과

```
공식 dw, db : [-51.166  -15.2965]
수치 dw, db : [-51.166  -15.2965]
```

공식과 수치 미분 결과가 소수점 넷째 자리까지 같습니다. (b 에 대한 공식 db 는 6장에서 유도합니다.) w = 1 에서 dw 가 음수이므로, 2장의 표대로 w 를 키워야 한다는 것도 알 수 있습니다.

핵심 포인트

- $\partial L/\partial w = (2/N)\Sigma(\hat{y} - y)\cdot x = 2\cdot MEAN((\hat{y} - y)\cdot x)$
- 겉 미분 2(·) × 속 미분 x. 오차 × 입력의 평균.

확인 문제

Q1. 데이터 1개의 제곱 오차 $(wx + b - y)^2$ 를 w 로 편미분하시오.
Q2. 데이터 (x, y) = (1, 5), (2, 8), 현재 w = 2, b = 1 이다. $\hat{y}$, 오차 $\hat{y} - y$, $\partial L/\partial w$ 를 구하고 w 를 키워야 할지 줄여야 할지 답하시오.

CHAPTER 5

학습률과 업데이트

영상 구간 **09:30~12:50** learning rate 를 곱하는 이유, 상수 2 를 학습률에 포함

학습 목표

- 학습률(learning rate)의 역할을 설명할 수 있다.
- 영상의 업데이트 식 LR · MEAN((ŷ - y)·x) 를 이해한다.

구한 그래디언트를 w 에서 **그대로** 빼지는 않습니다. 대신 0.01, 0.05 처럼 작은 수를 곱해서 조금만 움직입니다. 이 수를 **학습률(learning rate, LR, η)**이라고 합니다.

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

영상에서 말하듯 지금은 w 가 하나지만, 실제 모델은 파라미터가 매우 많고 업데이트를 수없이 반복합니다. 학습률로 한 번에 움직이는 **폭을 줄여 미세 조정**이 일어나게 하는 것입니다. 학습률은 사람이 정해 주는 아주 중요한 **하이퍼파라미터**입니다. (너무 크거나 작으면 어떻게 되는지는 8장에서 실험합니다.)

상수 2 는 학습률에 포함시킨다

η·(2/N)Σ... 에서 η 도 상수, 2 도 상수입니다. 상수끼리의 곱은 그냥 하나의 상수이므로, 영상에서는 **2 를 학습률에 포함시켜 지워** 버립니다. (예: η = 0.05 와 2 를 곱한 0.1 을 그냥 새로운 학습률로 보는 것)

$$\eta \cdot \frac{2}{N}\sum(\hat{y}-y)\,x \quad \Longrightarrow \quad \mathrm{gd}_w = \mathrm{LR}\cdot\mathrm{MEAN}((\hat{y}-y)\cdot x)$$

$$w \leftarrow w - \mathrm{gd}_w$$

주의

- 2 를 지운 식은 '정확한 기울기 × 학습률' 과 비교하면 **학습률이 절반**인 것과 같습니다. 방향은 똑같으므로 학습은 문제없이 됩니다. 다만 다른 책의 코드(2 를 남긴 식)와 같은 학습률 숫자를 쓰면 속도가 다를 수 있다는 점만 기억하세요.

핵심 포인트

- 학습률 = 한 걸음의 크기. 작게 곱해서 조금씩 움직인다.
- gd_w = LR · MEAN((ŷ - y)·x), w ← w - gd_w

확인 문제

Q1. 4장 Q2 와 같은 상황(데이터 (1, 5), (2, 8), w = 2, b = 1)에서 LR = 0.1 일 때 영상 방식의 gd_w 와 새 w 를 구하시오.
Q2. 그래디언트에 학습률을 곱하지 않고 그대로 빼면 어떤 문제가 생길 수 있는가?

CHAPTER 6

b 에 대한 편미분

영상 구간 **12:50~15:10** 같은 방법으로 b 의 그래디언트, b 는 1 로 바뀐다

학습 목표

- ∂L/∂b 를 유도할 수 있다.
- b 의 업데이트 식을 쓸 수 있다.

방법은 4장과 완전히 같습니다. 바깥 껍데기는 똑같이 2(wx + b - y), 달라지는 것은 **속 미분**뿐입니다. b 로 미분하면 wx 와 y 는 상수라 사라지고 **b 는 1** 이 됩니다.

$$\frac{\partial}{\partial b}(wx + b - y) = 1$$

$$\frac{\partial L}{\partial b} = \frac{1}{N}\sum 2(wx + b - y)\cdot 1 = \frac{2}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i) = 2\cdot \mathrm{MEAN}(\hat{y} - y)$$

w 와 마찬가지로 2 를 학습률에 포함시키면 영상의 최종 식이 됩니다. 곱하기 1 은 써 줄 필요가 없으니 지웁니다.

$$\mathrm{gd}_b = \mathrm{LR}\cdot \mathrm{MEAN}(\hat{y} - y) \qquad\qquad b \leftarrow b - \mathrm{gd}_b$$

의미: b 는 직선 전체를 위아래로 옮기는 값입니다. 예측이 평균적으로 실제보다 크면(오차 평균 > 0) b 를 줄이고, 작으면 b 를 키웁니다. 직관과 딱 맞죠.

핵심 포인트

- ∂L/∂b = 2·MEAN(ŷ - y). 속 미분이 1 이라 x 가 곱해지지 않는다.
- gd_b = LR · MEAN(ŷ - y), b ← b - gd_b

확인 문제

Q1. 4장 Q2 와 같은 상황(데이터 (1, 5), (2, 8), w = 2, b = 1), LR = 0.1 일 때 gd_b 와 새 b 를 구하시오.
Q2. ∂L/∂w 에는 x 가 곱해지는데 ∂L/∂b 에는 곱해지지 않는 이유를 '속 미분'으로 설명하시오.

CHAPTER 7

최종 정리와 파이썬 구현

영상 구간 **15:10~끝** 최종 업데이트 식 정리 (파이썬 코드는 다음 영상 예고 → 이 교안에서 구현)

학습 목표

- 경사하강법 업데이트 식 두 줄을 외워서 쓸 수 있다.
- 식을 그대로 파이썬 코드로 옮길 수 있다.

최종 정리 (영상 판서)

$$\mathrm{gd}_w = \mathrm{LR} \cdot \mathrm{MEAN}[(\hat{y} - y) \cdot x] \qquad w \leftarrow w - \mathrm{gd}_w$$
$$\mathrm{gd}_b = \mathrm{LR} \cdot \mathrm{MEAN}[\hat{y} - y] \qquad b \leftarrow b - \mathrm{gd}_b$$

수식	파이썬 (numpy)
ŷ = wx + b	`Y_hat = w * X + b`
ŷ - y	`error = Y_hat - Y`
LR · MEAN((ŷ - y)·x)	`gd_w = lr * np.mean(error * X)`
LR · MEAN(ŷ - y)	`gd_b = lr * np.mean(error)`
w ← w - gd_w, b ← b - gd_b	`w = w - gd_w, b = b - gd_b`

실습 코드 7-1 - 경사하강법으로 y = 3x + 2 찾기

```
w, b = 0.0, 0.0          # 아무 값에서 출발
lr = 0.05                # 학습률 (learning rate)

for epoch in range(1, 501):
    Y_hat = w * X + b                      # 1) 예측
    error = Y_hat - Y                      # 2) 오차 (y hat - y)
    gd_w = lr * np.mean(error * X)         # 3) LR * MEAN((y hat - y) * x)
    gd_b = lr * np.mean(error)             #    LR * MEAN(y hat - y)
    w = w - gd_w                           # 4) 업데이트
    b = b - gd_b
    if epoch in (1, 10, 50, 100, 500):
        print(f"epoch {epoch:3d} | MSE = {np.mean(error**2):8.4f} | w = {w:.4f}, b = {b:.4f}")
```

실행 결과

```
epoch   1 | MSE = 128.0895 | w = 1.7757, b = 0.5222
epoch  10 | MSE =   0.3907 | w = 3.2819, b = 1.0595
epoch  50 | MSE =   0.2622 | w = 3.1859, b = 1.3995
epoch 100 | MSE =   0.1955 | w = 3.1075, b = 1.6724
epoch 500 | MSE =   0.1561 | w = 2.9876, b = 2.0900
```

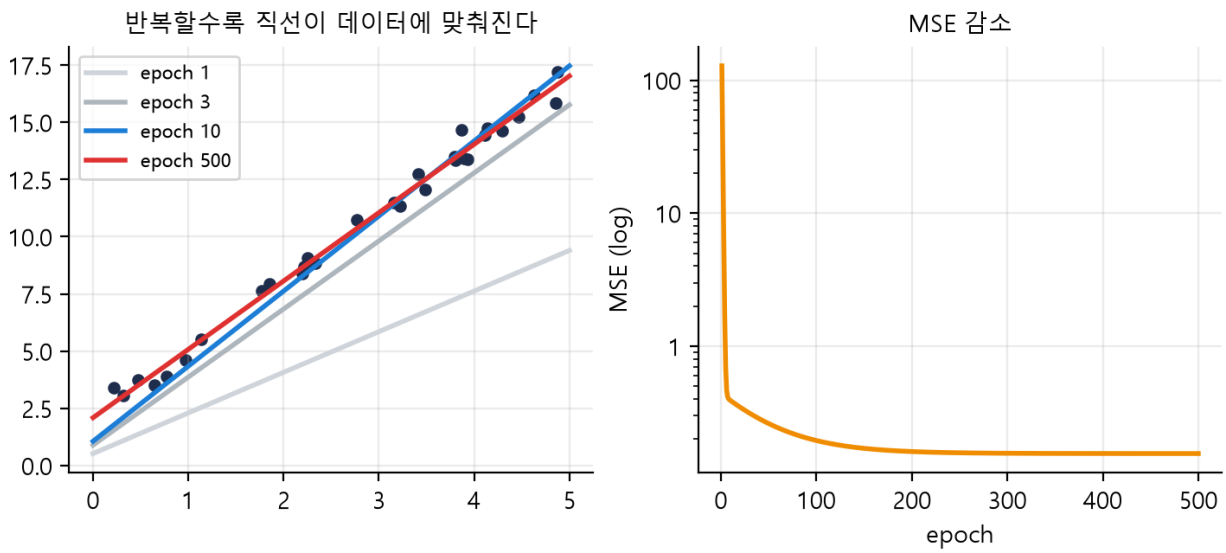


그림 7-1. 왼쪽: epoch 이 지날수록 직선이 데이터에 맞춰진다. 오른쪽: MSE 가 빠르게 줄어든다

w = 0, b = 0 에서 출발해 500 번 반복하자 w ≒ 2.99, b ≒ 2.09 가 되었습니다. 데이터를 만든 규칙 y = 3x + 2 에 매우 가깝습니다. (데이터에 잡음이 섞여 있어 정확히 3, 2 가 되지는 않습니다. 이 데이터에서 가장 좋은 값은 8장에서 확인합니다.) 출력되는 MSE 는 그 epoch 에서 업데이트하기 **전**의 값입니다.

epoch 은 전체 데이터를 한 번 다 보고 업데이트하는 것을 한 번으로 세는 단위입니다.

핵심 포인트

- 경사하강법 = 예측 → 오차 → 그래디언트 → 업데이트 의 반복.
- 수식 두 줄이 코드 두 줄로 그대로 옮겨진다.

확인 문제

Q1. 5·6장 문제에서 구한 새 값(w = 2.4, b = 1.25)으로 데이터 (1, 5), (2, 8) 의 MSE 를 다시 구하고, 업데이트 전(w = 2, b = 1)의 MSE 와 비교하시오.

Q2. 코드에서 `w = w - gd_w` 를 실수로 `w = w + gd_w` 로 쓰면 어떻게 될까?

CHAPTER 8

학습률 실험과 최소제곱법 (보충)

영상 구간 - 영상에 없는 보충 내용: 5장에서 '중요한 파라미터'라고 한 학습률을 직접 바꿔 보기

학습 목표
- 학습률이 너무 작거나 클 때 생기는 현상을 설명할 수 있다.
- 경사하강법 결과가 최소제곱법 결과와 같아짐을 확인한다.

8.1 학습률에 따라 달라지는 걸음

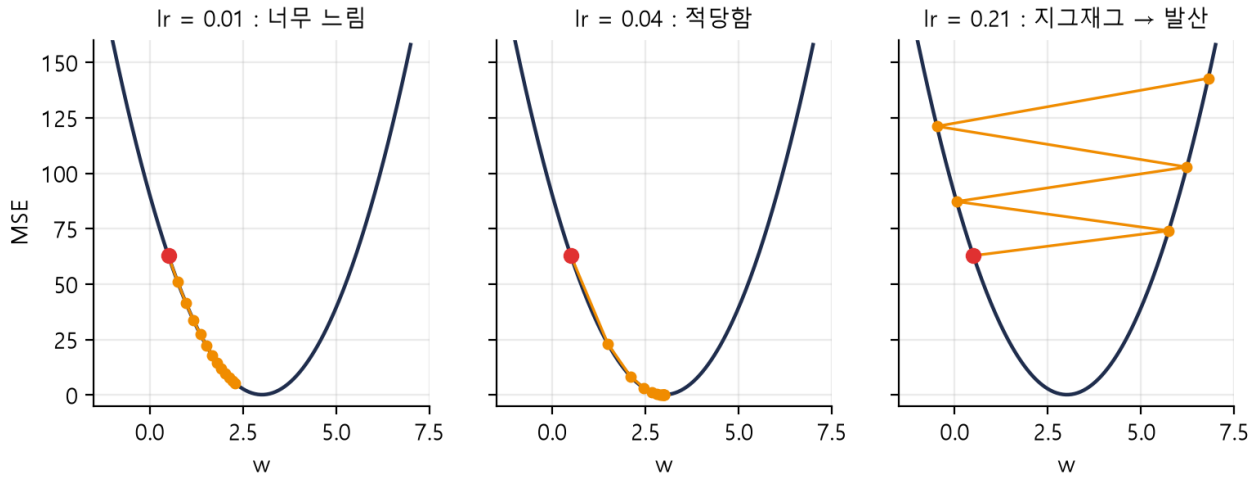


그림 8-1. 같은 시작점(빨간 점)에서 학습률만 바꿨을 때 w 가 움직이는 경로 (b = 2 고정)

학습률	현상
너무 작다	한 걸음이 작아 바닥까지 **아주 오래** 걸린다. 정해진 횟수 안에 도착하지 못한다
적당하다	몇 번 만에 바닥 근처에 도착하고 안정적으로 멈춘다
너무 크다	바닥을 **지나쳐** 반대편 벽으로 튄다. 점점 더 높이 튀면 손실이 폭발하는 **발산**이 일어난다

실습 코드 8-1 - 학습률별 200 epoch 학습 결과

```
def train(lr, epochs=200):
    w, b = 0.0, 0.0
    for _ in range(epochs):
        error = (w * X + b) - Y
        w -= lr * np.mean(error * X)
        b -= lr * np.mean(error)
    return w, b, mse(w, b)

for lr in [0.001, 0.01, 0.05, 0.1, 0.25]:
    w, b, loss = train(lr)
    print(f"lr = {lr:<6} | w = {w:10.4g}, b = {b:10.4g} | MSE = {loss:.4g}")
```

실행 결과

```
lr = 0.001  | w =      2.919, b =     0.8862 | MSE = 2.118
lr = 0.01   | w =      3.207, b =      1.326 | MSE = 0.2834
lr = 0.05   | w =      3.031, b =       1.94 | MSE = 0.1614
lr = 0.1    | w =      2.991, b =      2.077 | MSE = 0.1561
lr = 0.25   | w = -5.929e+45, b = -1.703e+45 | MSE = 4.084e+92
```

lr = 0.001 은 아직 b 가 0.89 로 정답과 멀고, lr = 0.1 은 잘 수렴했으며, lr = 0.25 는 w, b 가 -10^{45} 수준으로 폭발했습니다. 학습률을 정할 때는 작은 값에서 시작해 손실이 잘 줄어드는지 확인하며 조절합니다.

8.2 최소제곱법(OLS)과 비교

선형 회귀는 $\partial L/\partial w = 0$, $\partial L/\partial b = 0$ 을 연립방정식으로 풀어 바닥을 **한 번에** 구할 수도 있습니다. 이것이 **최소제곱법(OLS)**이며, numpy 의 `np.polyfit` 이 이 방법으로 직선을 구합니다.

실습 코드 8-2 - 경사하강법 vs 최소제곱법

```
w_gd, b_gd, _ = train(0.05, epochs=2000)
w_ls, b_ls = np.polyfit(X, Y, 1)            # 최소제곱법(OLS)으로 한 번에 계산

print(f"경사하강법 : w = {w_gd:.4f}, b = {b_gd:.4f}")
print(f"최소제곱법 : w = {w_ls:.4f}, b = {b_ls:.4f}")
```

실행 결과

```
경사하강법 : w = 2.9853, b = 2.0981
최소제곱법 : w = 2.9853, b = 2.0981
```

충분히 반복한 경사하강법이 최소제곱법과 소수점 넷째 자리까지 같은 답을 냈습니다. 경사하강법이 정말로 손실 함수의 바닥을 찾아간다는 증거입니다. 그런데 왜 굳이 경사하강법을 쓸까요? 딥러닝처럼 모델이 복잡하면 '한 번에 푸는 공식'이 없기 때문입니다. 경사하강법은 **미분만 할 수 있으면 어떤 모델에도** 쓸 수 있습니다.

핵심 포인트

- 학습률: 작으면 느리고, 크면 발산.
- 경사하강법의 도착점 = 최소제곱법의 해 (선형 회귀의 경우).

확인 문제

Q1. 학습률이 너무 크면 왜 발산하는지 그림 8-1 오른쪽을 보고 설명하시오.
Q2. 선형 회귀는 최소제곱법으로 한 번에 풀 수 있는데도 경사하강법을 배우는 이유는 무엇인가?

APPENDIX A

확인 문제 정답 및 해설

1장

Q1. 오차 $\hat{y} - y = [1, -1, 1]$, 오차의 합 = 1, SSE = 1 + 1 + 1 = 3, MSE = 3/3 = 1, MAE = 3/3 = 1

Q2. SSE 는 데이터 개수가 많을수록 커져서 데이터 수가 다른 경우 비교가 어렵다. 평균을 내면 개수와 상관없이 '한 데이터당 평균 오차'로 비교할 수 있고 값도 적당한 크기가 된다.

2장

Q1. 기울기가 양수이므로 w 를 줄여야 한다 (w ← w - 기울기).

Q2. -5 인 점. 그릇 모양(2차 함수)에서는 바닥에서 멀어질수록 경사가 가팔라지므로 기울기의 크기가 큰 점이 바닥에서 더 멀다. 둘 다 음수이므로 바닥의 왼쪽에 있다.

3장

Q1. 데이터 x, y (와 개수 N, 그리고 w 로 편미분할 때는 b 도 상수 취급)

Q2. $\hat{y} = 3 \cdot 2 + 0 = 6$, $(\hat{y} - y)^2 = (6 - 7)^2 = 1$

4장

Q1. $2(wx + b - y) \cdot x$

Q2. $\hat{y} = [3, 5]$, 오차 $\hat{y} - y = [-2, -3]$, (오차 × x) = [-2, -6], 평균 = -4, $\partial L/\partial w = 2 \cdot (-4) = -8$. 음수이므로 w 를 키워야 한다.

5장

Q1. gd_w = 0.1 × MEAN([-2, -6]) = 0.1 × (-4) = -0.4, 새 w = 2 - (-0.4) = 2.4

Q2. 한 번에 너무 크게 움직여 바닥을 지나쳐 버리고, 심하면 손실이 점점 커지며 발산할 수 있다. 학습률을 곱해 조금씩 움직여야 안정적으로 바닥에 다가간다.

6장

Q1. gd_b = 0.1 × MEAN([-2, -3]) = 0.1 × (-2.5) = -0.25, 새 b = 1 - (-0.25) = 1.25

Q2. 겉 미분 $2(wx + b - y)$ 는 같지만, 속 미분이 w 로 미분하면 x, b 로 미분하면 1 이기 때문이다.

7장

Q1. 업데이트 전: $\hat{y} = [3, 5]$, 오차 [-2, -3], MSE = (4 + 9)/2 = 6.5. 업데이트 후: $\hat{y} = [3.65, 6.05]$, 오차 [-1.35, -1.95], MSE = (1.8225 + 3.8025)/2 = 2.8125. 한 번의 업데이트로 손실이 줄었다.

Q2. 기울기 방향(오르막)으로 이동하는 '경사상승'이 되어 손실이 계속 커지고 w, b 가 발산한다.

8장

Q1. 기울기에 학습률을 곱한 걸음이 너무 커서 바닥을 지나 반대편의 더 높은 곳에 착지한다. 그곳은 경사가 더 가팔라 다음 걸음은 더 커지고, 이것이 반복되며 손실이 폭발한다.

Q2. 딥러닝처럼 복잡한 모델은 바닥을 한 번에 구하는 공식이 없거나 계산량이 너무 많다. 경사하강법은 미분(그래디언트)만 구할 수 있으면 어떤 모델에도 적용할 수 있다.

APPENDIX
B

공식 한눈에 보기

구분	공식	메모
모델	$\hat{y} = wx + b$	w 가중치, b 편향
오차	$\hat{y} - y$	부호가 있어 그냥 더하면 상쇄
손실 (MSE)	$L = \frac{1}{N}\sum (wx_i + b - y_i)^2$	변수는 w, b
w 의 기울기	$\frac{\partial L}{\partial w} = \frac{2}{N}\sum (\hat{y}_i - y_i)\, x_i$	겉 미분 × 속 미분 x
b 의 기울기	$\frac{\partial L}{\partial b} = \frac{2}{N}\sum (\hat{y}_i - y_i)$	속 미분 1
업데이트	$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$	기울기 반대 방향
영상 표기 (w)	$\mathrm{gd}_w = \mathrm{LR} \cdot \mathrm{MEAN}[(\hat{y} - y)\, x], \quad w \leftarrow w - \mathrm{gd}_w$	2 는 LR 에 포함
영상 표기 (b)	$\mathrm{gd}_b = \mathrm{LR} \cdot \mathrm{MEAN}[\hat{y} - y], \quad b \leftarrow b - \mathrm{gd}_b$	2 는 LR 에 포함

참고

원본 영상: 테디노트 TeddyNote, 「경사하강법 (Gradient Descent)의 기본 수식에 대해 알려드립니다」 (17분 42초, 2020.05.13)
https://www.youtube.com/watch?v=GEdLNvPIbiM
이 교안은 영상의 판서 흐름을 따라 정리하고, 그림 · 실습 코드 · 확인 문제와 8장(보충)을 덧붙였습니다.